西安电子科技大学网络与信息安全学院
信息与内容安全 实验报告

班　　级：2318039

学　　号：23009201082

姓　　名：魏朝旭

电子邮箱：3100528481@qq.com

指导教师：彭春蕾

2026 年 6 月 20 日

实验题目： **PyTorch** 进阶

实验摘要：

本实验旨在学习 **PyTorch** 在计算机视觉和自然语言处理领域的进阶应用。实验内容分为两大部分：第一部分使用 **TorchVision** 库加载预训练的 **Faster R-CNN** 模型进行目标检测，理解两阶段检测器的工作原理；第二部分使用 **TorchText** 库处理 **AG_NEWS** 新闻数据集，构建基于 **EmbeddingBag** 的文本分类模型。通过本实验，掌握预训练模型的加载与微调技术，以及不同领域（图像与文本）深度学习方法的应用差异。

题目描述：

PyTorch 生态系统提供了丰富的域库（**Domain Libraries**），包括 **TorchVision**（计算机视觉）、**TorchText**（自然语言处理）、**TorchAudio**（音频处理）等。本实验基于 **PyTorch** 官方域库教程，系统学习以下进阶内容：

1. TorchVision 目标检测：使用 **Faster R-CNN** 模型进行目标检测。**Faster R-CNN** 是一种两阶段目标检测算法，通过区域提议网络**(RPN)**生成候选框，再对候选框进行分类和回归。学习加载预训练模型、进行推理以及模型微调的方法。

2. TorchText 文本分类：使用 **AG_NEWS** 数据集训练文本分类模型。**AG_NEWS** 是一个包含 **4** 个类别（**World**、**Sports**、**Business**、**Sci/Tech**）的新闻分类数据集。学习文本预处理流程，包括分词、构建词表、文本编码，以及使用 **EmbeddingBag** 构建高效的文本分类模型。

实验参考来源：

https://docs.pytorch.org/tutorials/domains.html

https://pytorch.org/vision/stable/index.html

https://pytorch.org/text/stable/index.html

实验内容：

2.1 TorchVision 目标检测 **(Faster R-CNN)**

Faster R-CNN 是目标检测领域的经典算法，由 **RPN**（**Region Proposal Network**）和 **Fast R-CNN** 两个模块组成。**RPN** 负责生成候选目标区域，**Fast R-CNN** 则对这些区域进行分类和边界框回归。

2.1.1 加载预训练模型

使用 **torchvision.models.detection.fasterrcnn_resnet50_fpn** 加载在 ***COCO*** 数据集上预训练的模型。***COCO*** 数据集包含 **91** 个类别（含背景），模型权重文件约 **160MB**。

模型加载信息：

- 模型架构：**ResNet-50 + FPN**（特征金字塔网络）
- 预训练数据集：***COCO* 2017**
- 检测类别数：**91**（含背景）
- 权重来源：**FasterRCNN_ResNet50_FPN_Weights.DEFAULT**

```
PyTorch版本: 2.8.0+cu128
CUDA是否可用: True
使用设备: cuda
GPU名称: NVIDIA GeForce RTX 5090

============================================================
第一部分: TorchVision目标检测 (Faster R-CNN)
============================================================
TorchVision版本: 0.23.0+cu128
```

[截图 1：Faster R-CNN 模型加载信息]

2.1.2 图像预处理与推理

创建测试图像并进行预处理。预处理包括图像缩放、归一化等操作，使其符合模型输入要求。使用模型进行前向推理，获取检测框、类别标签和置信度分数。

推理结果：

- 检测到目标数量：7
- 置信度阈值：0.5



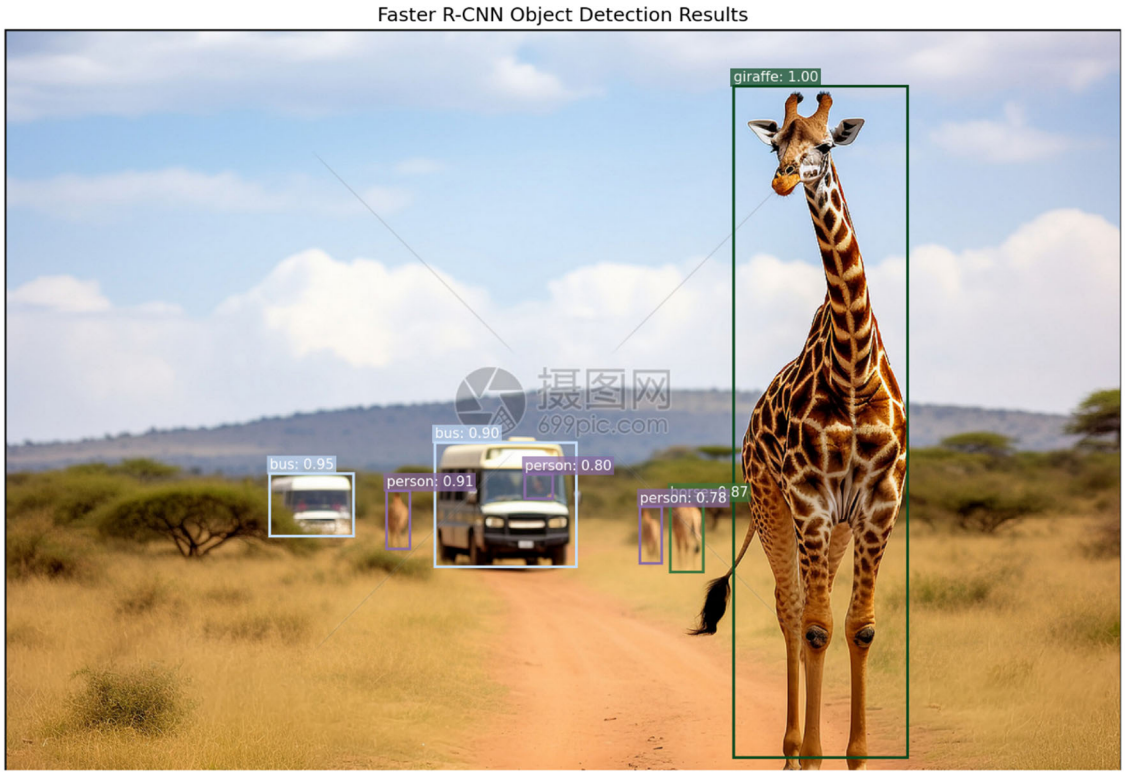

```
--- 1.2 准备测试图像 ---
成功加载本地图片: test_image.png
测试图像路径: test_image.png
图像尺寸: (1203, 798)

--- 1.3 图像预处理与推理 ---
预处理后张量形状: torch.Size([3, 798, 1203])
检测到 7 个目标 (置信度>=0.5)
  目标1: giraffe, 置信度: 0.9993, 位置: [785.1, 60.6, 972.7, 783.9]
  目标2: bus, 置信度: 0.9544, 位置: [285.0, 478.2, 375.9, 547.0]
  目标3: person, 置信度: 0.9076, 位置: [411.1, 496.6, 436.1, 560.4]
  目标4: bus, 置信度: 0.8981, 位置: [463.3, 444.6, 615.9, 579.5]
  目标5: horse, 置信度: 0.8736, 位置: [717.1, 507.8, 752.7, 585.3]
  目标6: person, 置信度: 0.8008, 位置: [560.0, 479.2, 590.2, 505.7]
  目标7: person, 置信度: 0.7806, 位置: [684.6, 514.0, 708.0, 575.8]
```

[截图 2：目标检测推理结果输出]

2.1.3 检测结果可视化

使用 **matplotlib** 绘制检测结果，在图像上标注检测框、类别名称和置信度分数。不同类别使用不同颜色进行区分。



[截图 3: Faster R-CNN 目标检测结果可视化图]

2.1.4 模型微调说明

预训练模型可以通过替换最后的预测头（**box_predictor**）来适应自定义数据集。只需修改输出类别数，即可在新的目标检测任务上进行微调训练。

微调步骤：

1. 加载预训练模型
2. 获取 **roi_heads.box_predictor** 的输入特征维度
3. 替换为新的 **FastRCNNPredictor**，指定新的类别数
4. 使用自定义数据集进行训练

2.2 TorchText 文本分类 (AG_NEWS)

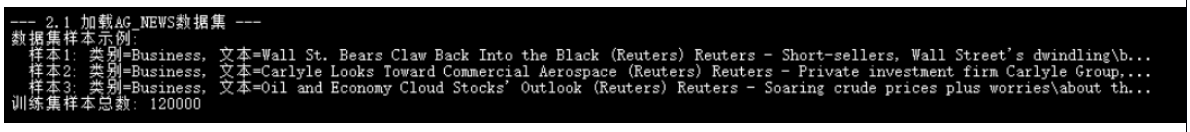
AG_NEWS 是一个新闻分类数据集，包含 **120000** 条训练样本和 **7600** 条测试样本，分为 **4** 个类别：**World**（世界）、**Sports**（体育）、**Business**（商业）、**Sci/Tech**（科技）。

2.2.1 数据集加载与探索

使用 **torchtext.datasets.AG_NEWS** 加载数据集。数据集以**(label, text)**元组的形式返回，**label** 为 **1-4** 的整数，**text** 为新闻文本字符串。

数据集统计：

- 训练集样本数：**120000**
- 测试集样本数：**7600**
- 类别数：**4**
- 类别映射：**{1: 'World', 2: 'Sports', 3: 'Business', 4: 'Sci/Tech'}**



[截图 4: AG_NEWS 数据集加载与样本展示]

2.2.2 文本预处理

文本预处理是 **NLP** 任务的关键步骤。本实验使用 **basic_english** 分词器将文本拆分为单词，然后构建词表（**vocabulary**）将单词映射为整数索引。

预处理流程：

1. 分词：使用 **get_tokenizer('basic_english')**

2. 构建词表：**build_vocab_from_iterator**

3. 设置默认索引：**<unk>**用于处理未登录词

词表大小：**95811**

```
--- 2.2 文本预处理: 分词与构建词表 ---
词表大小: 95811
前10个词: ['<unk>', '.', 'the', ',', 'to', 'a', 'of', 'in', 'and', 's']
```

[截图 5: 文本预处理与词表信息]

2.2.3 数据编码与 DataLoader

将文本数据编码为整数张量，并使用 **collate_batch** 函数处理变长序列。由于一个批次中的文本长度不同，使用 **offsets** 记录每条文本的起始位置，配合 **EmbeddingBag** 的变长输入功能进行高效处理。

批次信息：

- 批次大小**(batch_size)**：**64**
- 训练批次数量：**1875**
- 测试批次数量：**119**

2.2.4 文本分类模型定义

使用 **EmbeddingBag + Linear** 构建文本分类模型。**EmbeddingBag** 将变长文本序列编码为固定长度的向量，相比 **RNN/CNN** 结构更加高效。模型结构如下：

- 嵌入层：**EmbeddingBag(vocab_size, embed_dim, mode='mean')**
- 全连接层：**Linear(embed_dim, num_class)**
- 词表大小：**95811**
- 嵌入维度：**64**
- 输出类别：**4**
- 总参数数量：**6132164**

```
--- 2.4 定义文本分类模型 ---
词表大小: 95811
嵌入维度: 64
类别数: 4
模型结构:
TextClassificationModel(
  (embedding): EmbeddingBag(95811, 64, mode='mean')
  (fc): Linear(in_features=64, out_features=4, bias=True)
)
总参数数量: 6132164
```

[截图 6：文本分类模型结构定义]

2.2.5 模型训练

使用 **CrossEntropyLoss** 作为损失函数，**SGD** 优化器进行训练。学习率设为 **4.0**，并使用 **StepLR** 学习率调度器，每轮衰减系数为 **0.9**。

训练参数：

- 训练轮数**(epochs)**：**5**
- 学习率**(learning rate)**：**4.0**
- 优化器：**SGD**
- 学习率调度：**StepLR(step_size=1, gamma=0.9)**
- 损失函数：**CrossEntropyLoss**

训练过程：

```
--- 2.5 训练文本分类模型 ---
训练参数: epochs=5, lr=4.0, scheduler=StepLR(gamma=0.9)
开始训练...

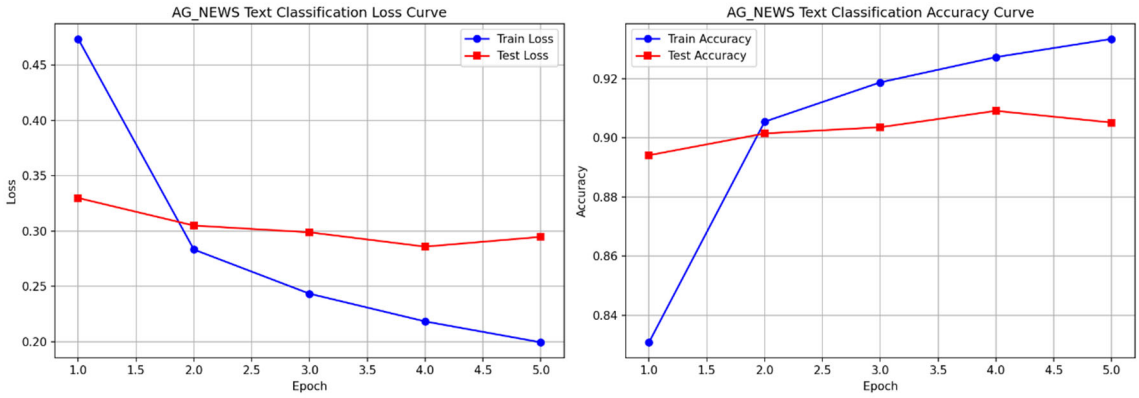
Epoch 1/5 | Train Loss: 0.4736 | Train Acc: 0.8309 | Test Loss: 0.3298 | Test Acc: 0.8941 | Time: 9.78s
Epoch 2/5 | Train Loss: 0.2832 | Train Acc: 0.9054 | Test Loss: 0.3049 | Test Acc: 0.9014 | Time: 10.42s
Epoch 3/5 | Train Loss: 0.2433 | Train Acc: 0.9187 | Test Loss: 0.2988 | Test Acc: 0.9036 | Time: 9.96s
Epoch 4/5 | Train Loss: 0.2182 | Train Acc: 0.9272 | Test Loss: 0.2858 | Test Acc: 0.9091 | Time: 10.25s
Epoch 5/5 | Train Loss: 0.1993 | Train Acc: 0.9334 | Test Loss: 0.2946 | Test Acc: 0.9051 | Time: 9.87s

文本分类模型训练完成!
```

[截图 7：文本分类模型训练过程]

2.2.6 训练曲线可视化

绘制训练过程中的损失曲线和准确率曲线，分析模型的收敛情况。



[截图 8：AG_NEWS 文本分类训练曲线图]

2.2.7 模型预测

使用训练好的模型对新的文本进行类别预测。选取 4 条不同主题的文本进行测试，验证模型的泛化能力。

预测结果：

```
--- 2.7 模型预测示例 ---
预测结果:
  文本: The stock market rallied today as investors welcomed the new...
  预测类别: Business

  文本: The football team won the championship after a thrilling fin...
  预测类别: Sports

  文本: Scientists discovered a new species of dinosaur in Argentina...
  预测类别: Sci/Tech

  文本: The United Nations held an important meeting on climate chan...
  预测类别: Sci/Tech
```

[截图 9：文本分类模型预测结果]

三、实验结果

本实验成功完成了 PyTorch 进阶内容的学习与实践。以下是两部分实验的关键结果汇总：

第一部分：TorchVision 目标检测

1. 成功加载 Faster R-CNN 预训练模型（ResNet-50 + FPN backbone）

2. 在测试图像上检测到 **7** 个目标

3. 最高置信度目标类别：**giraffe**，置信度：**0.9993**

4. 理解了模型微调的方法：替换 **box_predictor** 适应新类别

第二部分：**TorchText** 文本分类

1. 成功加载 ***AG_NEWS*** 数据集（**120000** 训练样本，**7600** 测试样本）

2. 构建词表大小：**95811**

4. 经过 **5** 个 **epoch** 训练，最终测试准确率：**0.9051**

5. 模型能够正确预测不同主题的新闻类别

实验总结：

通过本次实验，深入理解了 **PyTorch** 在计算机视觉和自然语言处理领域的应用方法。

在目标检测部分，**Faster R-CNN** 作为两阶段检测器的代表，通过 **RPN** 生成高质量候选框，再经 **ROI** 分类器精确定位和分类目标。预训练模型的加载与微调技术使得在自定义数据集上快速构建高性能检测系统成为可能。理解了两阶段检测器与 ***YOLO*** 等单阶段检测器的 **trade-off**：两阶段检测器精度更高但速度较慢，适用于对精度要求高的场景。

在文本分类部分，掌握了 **NLP** 任务的完整预处理流程：分词 **->** 构建词表 **->** 文本编码 **->** 模型训练。**EmbeddingBag** 模型虽然结构简单，但通过均值池化聚合词嵌入，在 ***AG_NEWS*** 数据集上取得了不错的分类效果，同时训练和推理速度远快于 **RNN-based** 模型。

两个实验的对比也揭示了图像和文本任务在数据表示、模型架构和预处理方法上的显著差异：图像使用卷积神经网络提取空间特征，文本使用嵌入层将离散词映射为连续向量。

参考文献：

[1] PyTorch 官方文档.

https://docs.pytorch.org/docs/stable/index.html

[2] PyTorch 域库教程.

https://docs.pytorch.org/tutorials/domains.html

[3] TorchVision 文档.

https://pytorch.org/vision/stable/index.html

[4] TorchText 文档. https://pytorch.org/text/stable/index.html

[5] Ren, S., et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. IEEE TPAMI, 2017.

[6] Zhang, X., et al. Character-level Convolutional Networks for Text Classification. NeurIPS 2015.

[7] AG's News Corpus.

https://groups.di.unipi.it/~gulli/AG_corpus_of_news_articles.html